

Daemon Conversions in Distributed Self-Stabilizing Algorithms

Wayne Goddard¹, and Pradip K. Srimani¹

School of Computing, Clemson University, Clemson, SC 29634-0974

Abstract. We consider protocols to transform a self-stabilizing algorithm for one daemon to one that can run under a different daemon. In the literature, there are several daemons, and several possible attributes of those daemons, and it is customary to detail the choice of daemon one is using in designing a specific self-stabilizing algorithm. The choice of daemon plays an important role in designing self-stabilizing algorithm in terms of correctness and convergence time analysis; techniques and complexity vary widely with the type of daemons used. In order to simplify algorithm development in a systematic way, it would be useful to have to consider only one “canonical” daemon and then to use a relatively mechanical procedure to convert the algorithm to any other daemon when needed. We give the first (full) proof that, provided there are IDs, any algorithm that self-stabilizes only under a fair central daemon can be converted to one that self-stabilizes under an unfair read/write daemon.

1 Introduction

A self-stabilizing algorithm is a distributed algorithm that is designed to converge to a desired state without coordination or initialization [1]. Each node participates in the distributed algorithm based on local knowledge: its own state and the states of its immediate neighbors. The objective is to achieve some global objective – a predicate defined on the local states of all the nodes in the network – based on local actions where individual nodes have no global knowledge about the network. In order to analyze the correctness of the algorithm and its time complexity, a *daemon* is assumed: the daemon plays the role of both scheduler and adversary. In the literature, there are several daemons, and several possible attributes of those daemons, and it is customary to detail the choice of daemon one is using in designing a specific self-stabilizing algorithm. Indeed, the choice of daemon plays an important role in designing self-stabilizing algorithm in terms of correctness and convergence time analysis; techniques and complexity vary widely with the type of daemons used.

In order to simplify algorithm development in a systematic way, it would be useful to have to consider only one “canonical” daemon and then to use a relatively mechanical procedure to convert the algorithm to any other daemon if need be. One of the goals of this paper is to show that, for a price (either slowdown of the convergence time or additional storage requirement at the nodes), one can restrict one’s attention to a specific daemon, at least when the nodes

in the network are assigned unique IDs. In general, we consider the problem of converting self-stabilizing algorithms that run under one daemon to work under another daemon; these are called *transforms* [2–5]. Our purpose in this paper is to do a systematic investigation of all possible transforms between different daemons. We provide new transforms and improve upon some of the existing schemes. We give the first (full) proof that, provided there are IDs, any algorithm that self-stabilizes only under a fair central daemon can be converted to one that self-stabilizes under an unfair read/write daemon.

1.1 Definitions and Terminology

For this paper, we work in the *shared-variable* or *state-reading* model in which a node can directly read its neighbors’ variables. We restrict attention to undirected, bidirectional links. All computation is deterministic unless otherwise stated. One can look at [6, 7] for a general overview of the paradigm of self-stabilization and its requirements.

A self-stabilizing algorithm is usually written as a collection of production *rules* at each node: each rule specifies a condition and an action. The *condition* is a boolean predicate on the state of the node and the states of its neighbors; the action or *move* is a change in the state of the node executing the action. A node is *privileged* at a particular time if the condition of one or more of its rules is satisfied. Note that a node might stop being privileged if a neighbor moves.

The *central daemon* (sometimes called a serial daemon) chooses or *taps* exactly one privileged node to move at each step. In contrast, the *distributed daemon* taps a nonempty subset of the privileged nodes to move at each step. These daemons are considered adversarial. A special case of the distributed daemon is the *synchronous daemon*: under this scheduler, at every step every privileged node moves.

One can also consider the granularity of the computation at a node. In the *coarse* daemon, all computation by a node is completed in one atomic step. This is the most common assumption in self-stabilization. Dolev et al. [8] proposed a self-stabilizing version of *read/write atomicity*. In this, a node’s action is broken down into two atomic steps: (i) reading the state of all its neighbors, and (ii) updating its state. That is, each rule executes in a two-phase fashion: the read phase and the write phase. Whenever a node is tapped by the daemon, it reads the states of its neighbors and remembers the information; when it is tapped for the second time it executes the rule and changes its own variables. Since the daemon can choose an arbitrary gap between the read- and write-steps, the update might be made on stale information. The read/write daemon is defined to be a distributed daemon with read/write atomicity. See, e.g. [9] or [8].

In another direction, there is the concept of a fair and unfair daemon. For a (weakly) *fair daemon*, every node that is continuously privileged is tapped eventually. For an *unfair* daemon, there is no such restriction. See [6] for details. An algorithm running under fair daemon is thus expected to converge faster than

when running under an unfair daemon: an unfair daemon represents a worst-case analysis of the algorithm. While it is to some extent a theoretical device, an unfair daemon does cover a situation where some node is “frozen” because of conditions external to the program.

In yet another direction of classification of the daemons, one may assume that the nodes in the network have unique **IDs**. If this is not assumed we say that the algorithm is **anonymous**. Note that in case of anonymous algorithms, no node can explicitly use any node ID to make any decision or to take any action; we use node identifiers for reference purpose only. A self-stabilizing algorithm is said to be **silent** if it terminates; that is, it is guaranteed to reach a state where no node is privileged. See [10]. Some problems such as token circulation and clock synchronization are inherently not silent. Other problems such as leader election and maximal independent set may be silent, though some of the solutions to leader election are not (e.g., [11]).

We measure the running times of the algorithms in terms of the maximum number of **steps** needed for the algorithm to converge to a stable legitimate state in the worst case. Another notion used is that of **round**: this is a minimal interval during which every continuously privileged node is tapped.

2 Main Results

In this section we lay out the results of the paper. The proofs are provided in Section 3. We consider first the case that IDs are not available. Randomness can clearly be used to create IDs (see for example [12]) – and most daemon-conversion results assume IDs or create them. However, IDs require $\Omega(\log n)$ space, where n denotes the number of nodes. We have previously shown [13] that that one can convert a central-daemon algorithm to a distributed-daemon algorithm using only $O(1)$ extra storage per node:

Theorem 1 [13] *Any algorithm that self-stabilizes under an unfair central daemon can be converted to a randomized one that self-stabilizes under an unfair distributed daemon, using constant extra space, without IDs, and with at most $O(n^3)$ expected slowdown, while preserving silence.*

We provide a simple algorithm for the subproblem of **ensuring fairness**. As a consequence we have a simpler proof of the following result which can be extracted from the work of Beauquier et al. [14]:

Theorem 2 [14] *Any algorithm that self-stabilizes only under a fair central daemon can be converted to one that self-stabilizes under an unfair central daemon, without using IDs.*

This algorithm uses bounded memory, provided an upper bound on the size of the network is known. A significant open question is whether one can achieve this transform in bounded memory in an anonymous network **without any assumptions**. We give our alternative proof of Theorem 2 in Section 3.2.

It is essentially known, though to our knowledge not explicitly proven in the literature, that any algorithm that self-stabilizes under the central daemon using coarse atomicity can be converted to one that stabilizes under a read/write daemon provided there is symmetry breaking such as IDs or randomness. (See the next subsection for a discussion of the literature.) The problem that remains is to determine the price one pays in terms of slowdown in convergence time and/or additional storage need at the nodes. Our first result in this regard is that the price for conversion from central to distributed daemon is small.

Theorem 3 *Provided there are IDs, any algorithm that self-stabilizes under an unfair central daemon can be converted to one that self-stabilizes under an unfair distributed daemon, preserving silentness. For the general distributed daemon, the slowdown is linear (in the number of nodes). For the synchronous daemon, the slowdown is constant.*

Thus, for example, one immediately obtains linear-move synchronous algorithms for several problems like maximal matching and maximal independent set; linear-move algorithms under the central daemon are given in [15]. We prove Theorem 3 in Section 3.3. Our next result provides a conversion from central to read/write daemon, but the price seems higher.

Theorem 4 *Provided there are IDs, any algorithm that self-stabilizes under an unfair central daemon can be converted to one that self-stabilizes under an unfair read/write daemon, while preserving silentness.*

The **cross-over composition** idea of Beauquier et al. [16] is more powerful and general, but the resulting scheduling/synchronizing algorithm is not silent. Theorem 4 is proved in Section 3.4. Putting Theorems 2 and 4 together, we immediately obtain the most general result:

Theorem 5 *Provided there are IDs, any algorithm that self-stabilizes only under a fair central daemon can be converted to one that self-stabilizes under an unfair read/write daemon.*

2.1 Related Work

The concept of daemon can be thought of as two parts: The central daemon promises exclusivity, while the fair daemon promises each processor gets its turn. So to solve the problem of daemon conversion, one must show how to ensure (1) that no two adjacent nodes are ever simultaneously enabled for the original algorithm, and (2) that any node gets its turn after a while. The first of these is called local mutual exclusion, and the second is called fairness.

One solution to mutual exclusion is given by Lamport [17] (the bakery paper), and the ULME algorithm of [14] is largely the natural extension to general networks. Most of the papers on self-stabilizing mutual exclusion (see for example [9] or [6]) are essentially token-passing algorithms.

Fairness can also be thought of as a clock synchronization problem, and as such was investigated by Awerbuch et al. [18] and others. Their results hold in an anonymous network. If the processors have no global knowledge of the network, then these results use unbounded memory. If the processors have global knowledge (e.g. a bound on the order n or diameter d), then Awerbuch et al. provide a time-optimal (meaning $O(d)$ rounds given a fair daemon) if one also has IDs. Karaata [19] discusses ways of fairness enhancement.

There are several important related daemon-conversion results which address both fairness and mutual exclusion. Indeed, several of these results are deeper and more powerful than ours, but do not quite answer the questions we consider here. These include:

- Gouda and Haddix [20] provided an alternator which can be used to convert a central daemon algorithm to run under an unfair distributed daemon using IDs.
- Mizuno and Kakugawa [21] provided tools to convert a central daemon algorithm to run in a real distributed environment. They later reported on case studies [22]
- Beauquier et al. [14] provided a solution to local mutual exclusion which uses IDs and which also handles fairness, provided the initial algorithm can already handle a read/write daemon.
- Nesterenko and Arora [23] provided several results about conversion from one daemon to another, provided the daemon is fair. These results built on their solution to the dining philosophers problem.
- In the book [6], Dolev provides a conversion from central to read/write daemon, provided the daemon is fair and there is a distinguished processor.

Shukla et al. [24] provided a method using randomness that can be used to convert some central daemon algorithms to run under a distributed daemon.

3 The Transforms

We now prove Theorems 1–4. In each case we provide a transform from an algorithm $\mathcal{S}$ to an algorithm $\mathcal{S}'$. The algorithms are presented as the code for a node i . We define the boolean predicate

$$p_{\mathcal{S}}(i) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{node } i \text{ is privileged for algorithm } \mathcal{S} \text{ in a given system state} \\ 0 & \text{otherwise.} \end{cases}$$

The notation $N(i)$ denotes the set of neighbors of node i .

3.1 Central to Distributed without IDs

It has been shown [13] that randomization can be used to ensure local mutual exclusion between neighboring nodes, and so to convert a central daemon self-stabilizing algorithm to its distributed daemon equivalent. We briefly describe the algorithm and the result for the sake of completeness.

Let $\mathcal{S}$ be a self-stabilizing algorithm that works for an unfair, central, coarse daemon, but does not use IDs. In order to design the new algorithm $\mathcal{S}'$ to work for a distributed daemon, we add to each node i a boolean flag $b(i)$ in addition to the $\mathcal{S}$ -variables. This flag is designed to be true if the node is privileged for the underlying algorithm $\mathcal{S}$ and if the node is the only node in its neighborhood that has its flag set. When two adjacent nodes are simultaneously $\mathcal{S}$ -privileged and have their flag bits set, the nodes randomly determine a new value of their flag bits. A node can only execute the underlying algorithm if it is indeed privileged for $\mathcal{S}$ and is the only node in its neighborhood that has its flag set. The new algorithm $\mathcal{S}'$ is shown as Algorithm 1.

Algorithm 1 *Using randomness for exclusivity*

Variables: binary $b(i)$ (and variables needed for $\mathcal{S}$)

BitClear: if $b(i) = 1$ and not $p_{\mathcal{S}}(i)$

 then set $b(i) = 0$

BitSet: if $p_{\mathcal{S}}(i)$ and $b(i) = 0$ and $\forall j \in N(i) : b(j) = 0$

 then set $b(i) = 1$

BitToss: if $p_{\mathcal{S}}(i)$ and $b(i) = 1$ and $\exists j \in N(i)$ with $b(j) = 1$

 then set $b(i) = \text{RANDOM}$ (toss a fair coin to determine the new value of $b(i)$)

Step: if $p_{\mathcal{S}}(i)$ and $b(i) = 1$ and $\forall j \in N(i) : b(j) = 0$

 then execute one step of $\mathcal{S}$ at i

Under a distributed daemon, Algorithm 1 (i) achieves local exclusivity for $\mathcal{S}$, i.e., no two adjacent nodes execute the underlying algorithm $\mathcal{S}$ concurrently; (ii) cannot terminate while there is an $\mathcal{S}$ -privileged node.

3.2 Fair to Unfair Central Daemon

Beauquier et al. [14] showed how to enforce fairness using bounded variables, provided the nodes know a bound on the number of nodes in the graph. We give an alternative algorithm. Our result achieves fairness without having to wait for a period of stabilization: it is immediately self-stable. As in [14], the resulting algorithm is not silent. The algorithm is also similar to the alternator of Gouda and Haddix [20]. However, Algorithm 2 does not use IDs and achieves only fairness.

Given a self-stabilizing algorithm $\mathcal{S}$ that works for a fair central daemon, we define a new algorithm $\mathcal{S}'$ as follows. Each node maintains an additional **counter** variable $c(i)$ in the range $0 \dots n$ with wrap around (where n is the number of nodes in the network). All arithmetic is modulo $n + 1$. We define the Boolean predicate $p_{\mathcal{S}}(i)$ as before. A node is privileged for algorithm $\mathcal{S}'$ provided there does not exist a neighbor with a counter with the next higher value. If it is tapped, then a node executes $\mathcal{S}$ if $\mathcal{S}$ -privileged, and in any event, it increments its counter. The new algorithm $\mathcal{S}'$ is shown as Algorithm 2.

Algorithm 2 *Bounded counters for fairness*

Variables: counter $c(i)$ in the range $0 \dots n$ inclusive (and variables needed for $\mathcal{S}$)

Update: if $\exists j \in N(i)$ with $c(j) = c(i) + 1$

then (a) if $p_{\mathcal{S}}(i)$ then execute one step of $\mathcal{S}$ at i ; and

 (b) in any event, $c(i)++$ (modulo $n+1$)

Lemma 1 *Algorithm 2 is always live and ensures fairness for $\mathcal{S}$.*

Proof. Suppose the algorithm terminates. Consider any node i ; say $c(i) = 0$. It must have a neighbor, say j , with counter $c(j) = 1$. That node in turn must have a neighbor, say k , with counter $c(k) = 2$. And so on. But this implies $n+1$ different values of the counters, and hence $n+1$ different nodes, which is impossible. Thus, at every step there exists at least one node that is privileged for Algorithm 2.

As for fairness, consider a period of time when a node's counter goes from 0 to 0. There cannot be a neighbor that has not moved in that period, since each move precludes a neighbor with the next higher value. Indeed, if $M_i(t)$ denotes the number of moves of node i up until time t , then it always holds that $|M_i(t) - M_j(t)| \leq n$ for any pair i and j of adjacent nodes. Overall, there can be at most $O(n^2)$ other moves between two consecutive moves of the same node. Thus, as long as a node is $\mathcal{S}$ -privileged, it will get to move in at most $O(n^2)$ steps.

This establishes Theorem 2:

Theorem 2 [14] *Any algorithm that self-stabilizes only under a fair central daemon can be converted to one that self-stabilizes under an unfair central daemon, without using IDs, with at most $O(n^2)$ slowdown.*

Actually, nowhere in the proof did we use the fact that the daemon was central. So one can use this to ensure fairness for other daemons such as the distributed daemon. By combining the conversions of this subsection and the previous subsection, one can obtain a new local mutual exclusion algorithm; we omit the details.

3.3 Central to Distributed with IDs

We next consider the conversion to distributed daemon without introducing randomness. Note that we do not require fairness. The central daemon as a scheduler enforces that neighboring nodes do not move concurrently, that is, local mutual exclusion. So it suffices for us to achieve this under the distributed daemon. We use the standard idea of interpreting the IDs as priorities such as in work on philosopher problems or the MIS algorithm of Kakugawa et al. [21].

Let $\mathcal{S}$ be a self-stabilizing algorithm that works for an unfair, central, coarse daemon. We define a new algorithm $\mathcal{S}'$ as follows. We add to each node i a

Algorithm 3 *Using IDs for exclusivity*

Variables: binary $b(i)$ (and variables needed for $\mathcal{S}$)

BitUpdate: if $b(i) \neq p_{\mathcal{S}}(i)$

 then $b(i) = p_{\mathcal{S}}(i)$

Step: if $b(i) = 1$ and $p_{\mathcal{S}}(i)$ and $\nexists j \in N(i)$ with $b(j) = 1$ and $j < i$
 then execute one step of $\mathcal{S}$ at i , and set $b(i) = 0$

boolean flag $b(i)$. Algorithm 3 assumes that nodes are assigned unique IDs and has two rules as shown below.

The effect of Rule **BitUpdate** is that $b(i)$ contains the correct value of $p_{\mathcal{S}}(i)$. The effect of Rule **Step** is that a node i can enter execution of $\mathcal{S}$ only if its bit $b(i)$ is set and its ID is a local minimum amongst those neighbors which have their respective bits set.

Lemma 2 *Algorithm 3 under a distributed daemon achieves exclusivity for $\mathcal{S}$, i.e., no two adjacent nodes execute the underlying algorithm $\mathcal{S}$ concurrently.*

Proof. Consider two adjacent nodes i and j with $i < j$. For a node i to be able to execute Rule **Step** when it is tapped by the distributed daemon, at the point it reads the variables, it must have its b -bit set, and none of its smaller neighbors can have their b -bit set. Thus if node i is privileged for Algorithm 3 for Rule **Step** in a given system state, since $j > i$, the neighbor j , cannot be privileged for Algorithm 3 for Rule **Step**. Thus two adjacent nodes cannot execute Rule **Step** simultaneously.

Lemma 3 *If $\mathcal{S}'$ (Algorithm 3) terminates, then no node is privileged for $\mathcal{S}$.*

Proof. Assume that Algorithm 3 terminates. Then no node is privileged for Rule **BitUpdate**. So the bit $b(i) = p_{\mathcal{S}}(i)$ for each node i . The fact that no node is privileged for Rule **Step** then means that no node is $\mathcal{S}$ -privileged (consider the smallest $\mathcal{S}$ -privileged node; it must be privileged by Rule **Step**). Thus $\mathcal{S}$ has finished executing.

Lemma 4 *Algorithm 3 achieves progress in the original algorithm $\mathcal{S}$. In particular, the slowdown under the general distributed daemon is $O(n)$, and under the synchronous daemon is constant.*

Proof. Consider first the distributed daemon. The first rule can execute at most once per node between executions of Rule **Step**. Hence the slowdown is at most linear.

For the synchronous daemon, there cannot be two successive steps without Rule **Step** being executed. Hence the slowdown is at most constant. Thus,

Theorem 3 *Provided there are IDs, any algorithm that self-stabilizes under an unfair central daemon can be converted to one that self-stabilizes under an unfair distributed daemon, preserving silentness. For the general distributed daemon, the slowdown is at most $O(n)$. For the synchronous daemon, the slowdown is constant.*

3.4 Central to Read/Write with IDs

Algorithm 3 does not quite work for read/write atomicity, but we can modify the idea to handle read/write atomicity. In the worst case, this transform might lead to quadratic slowdown.

As before, let $\mathcal{S}$ be a self-stabilizing algorithm that works for an unfair, central, coarse daemon. In order to design the new algorithm $\mathcal{S}'$ to work for a read/write daemon, we add to each node i a boolean flag $b(i)$. This flag is designed to be true if the node is privileged for the underlying algorithm $\mathcal{S}$ and has higher priority (lower ID) than its waiting $\mathcal{S}$ -privileged neighbors. The node can only execute the underlying algorithm if it is indeed privileged for $\mathcal{S}$ and is the only node in its neighborhood that has its flag set. The new algorithm $\mathcal{S}'$ is shown as Algorithm 4.

Algorithm 4 *Using IDs for exclusivity under read/write*

Variables: binary $b(i)$ (and variables for $\mathcal{S}$)
Withdraw: if $b(i) = 1$ and not $p_{\mathcal{S}}(i)$
 then set $b(i) = 0$
Concede: if $b(i) = 1$ and $\exists j \in N(i)$ with $b(j) = 1$ and $j < i$
 then set $b(i) = 0$
Assert: if $p_{\mathcal{S}}(i)$ and $b(i) = 0$ and $\forall j \in N(i) : b(j) = 0$
 then set $b(i) = 1$
Step: if $p_{\mathcal{S}}(i)$ and $b(i) = 1$ and $\forall j \in N(i) : b(j) = 0$
 then execute one step of $\mathcal{S}$ at i

Remark 1. The daemon can tap a node to execute Rule Step immediately after Rule Assert, but we need the operation in two steps for coping with the read/write atomicity. Essentially, at the read-tap of the Rule Step, the node double-checks that every neighbor has bit clear. Even if one is in the process of setting its flag (write-tap for Rule Assert), that neighbor has to double-check (read-tap of Rule Step) before it executes write-phase of Rule Step and will be blocked.

Lemma 5 *If Algorithm 4 terminates, then no node is privileged for $\mathcal{S}$.*

Proof. Suppose at termination of Algorithm 4 there is a node i with $b(i) = 1$. Since node i is not privileged for Rule Withdraw, it is $\mathcal{S}$ -privileged. Further, since no node is privileged for Rule Concede, all neighbors of i have b -bits zero (if there is a node j , $j > i$ and $b(j) = 1$, node j must be privileged by Rule Concede). Then node i is $\mathcal{S}$ -privileged and all its neighbors have their b -bits zero; hence node i is privileged for Rule Step, a contradiction. Hence, at termination of Algorithm 4, each node i has $b(i) = 0$.

Since no node is privileged for Rule Assert, it follows that there is no $\mathcal{S}$ -privileged node. That is, if $\mathcal{S}'$ terminates, then $\mathcal{S}$ has also terminated.

Lemma 6 *Algorithm 4 achieves local mutual exclusion among neighbor nodes and faithfully executes the original algorithm $\mathcal{S}$.*

Proof. Consider a node i that executes Rule **Step**. We claim that between the read- and the write-steps for this move, no neighbor can start or finish Rule **Step**.

When node i performs the read-phase for Rule **Step**, it must have that $b(i) = 1$ and for all $j \in N(i)$, $b(j) = 0$; i.e., no neighbor j is currently executing Rule **Step**. Also, until node i completes the write-phase of Rule **Step**, the flag $b(i)$ remains set at 1 and hence no neighbor j of node i can be privileged for Rule **Step**. This also means that the step of $\mathcal{S}$ made by node i is based on correct (current) data.

Lemma 7 *Algorithm 4 achieves progress in the original algorithm $\mathcal{S}$; when $\mathcal{S}$ stabilizes, Algorithm 4 also stabilizes.*

Proof. Consider an interval T (series of moves made by Algorithm 4) during which the write-phase of Rule **Step** is not executed. Then the $\mathcal{S}$ -variables at each node i remain constant during this interval T , and so whether a node is $\mathcal{S}$ -privileged remains constant. (Predicate $p_{\mathcal{S}}(i)$ for each node i remains constant but $b(i)$ may change since some node(s) may still have stale read-data from before T started.)

We observe that during this interval T , each node i can execute either Rule **Withdraw** or Rule **Assert** at most once (since $p_{\mathcal{S}}(i)$ does not change); thus, there can be at most n moves during this interval T without any node executing Rule **Concede**. When a node i executes Rule **Concede**, the node i cannot execute Rule **Concede** again until the b -bit of node i or any of its neighbors is changed by execution of Rule **Withdraw** or Rule **Assert**. Also, when a node i executes Rule **Concede**, and no node changes its b -bit, none of the neighbors of node i can execute Rule **Concede**. Thus, when neither Rule **Withdraw** nor Rule **Assert** is executed by any node, the maximum number of times Rule **Concede** can be executed during the interval T is at most the size of the maximum independent set of the network graph; that is at most n , the number of nodes.

Hence, the maximum number of moves possible during the interval T is $O(n^2)$. Hence, if Rule **Step** is not executed—meaning the underlying algorithm $\mathcal{S}$ has stabilized—then the algorithm $\mathcal{S}'$ will eventually terminate.

The above three lemmas establish Theorem 4:

Theorem 4 *Provided there are IDs, any algorithm that self-stabilizes under an unfair central daemon can be converted to one that self-stabilizes under an unfair read/write daemon, while preserving silentness.*

4 Conclusion

The results in this paper reaffirm that in the deterministic ID-based shared-variable model, all daemons are equally powerful. An alternative to the shared-variable model is *link registers* where each pair of adjacent nodes have a register to pass messages. If the nodes have IDs, then link-registers can be trivially

simulated in general memory—a node writes the contents of each link-register next to each neighbor’s ID. Thus link-registers and shared-variable are equivalent in ID-based networks. It follows that all daemons are equally powerful in this model too.

An interesting question is of comparing the powers of different daemons in **deterministic anonymous networks**. Link-registers and a distinguished node or root allow one to form a breadth-first-search spanning tree and hence assign IDs; see e.g. [8]. Thus all daemons are equally powerful in this case. However, without a root, the results of [9, 25] and others on rings show that the distributed and central daemon have different powers. That is, there are some problems which have a solution under a central daemon but do not have a solution under a distributed daemon. This difference holds in both the link-register and shared-variable case. One way to proceed might be to determine exactly which problems have solutions. Angluin’s well-known arguments [26] about symmetry-breaking provide several limits on what problems can be solved, but are these essentially the only limits?

Acknowledgement:

The work was partially supported by NSF Awards CCF 0832582 and DBI-0960586.

References

1. E.W. Dijkstra. Self stabilizing systems in spite of distributed control. *Comm. ACM*, 17:643–644, 1974.
2. T. Herman. Models of self-stabilization and sensor networks. In *Distributed Computing—IWDC 2003, Kolkata, India, Samir R. Das, Sajal K. Das (Eds.) Springer LNCS:2918*, pages 205–214, 2003.
3. Swan Dubois and Sébastien Tixeuil. A taxonomy of daemons in self-stabilization. *CoRR*, abs/1110.0334, 2011.
4. Amos Korman, Shay Kutten, and Toshimitsu Masuzawa. Fast and compact self stabilizing verification, computation, and fault detection of an mst. In *Proceedings of the 30th annual ACM SIGACT-SIGOPS symposium on Principles of distributed computing*, PODC ’11, pages 311–320, New York, NY, USA, 2011. ACM.
5. Joffroy Beauquier, Sylvie Delat, and Sammy Haddad. A 1-strong self-stabilizing transformer. In Ajoy Datta and Maria Gradinariu, editors, *Stabilization, Safety, and Security of Distributed Systems*, volume 4280 of *Lecture Notes in Computer Science*, pages 95–109. Springer Berlin / Heidelberg, 2006.
6. S. Dolev. *Self-Stabilization*. MIT Press, 2000.
7. G. Tel. *Introduction to Distributed Algorithms*. Cambridge University Press, 1994.
8. S. Dolev, A. Israeli, and S. Moran. Self-stabilization of dynamic systems assuming only read/write atomicity. *Distrib. Comput.*, 7:3–16, 1993.
9. A. Israeli and M. Jalfon. Uniform self-stabilizing ring orientation. *Inform. Comput.*, 104:175–196, 1993.
10. S. Dolev, M.G. Gouda, and M. Schneider. Memory requirements for silent stabilization. In *PODC96 Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing*, pages 27–34, 1996.

11. G. Itkis, C. Lin, and J. Simon. Deterministic, constant space, self-stabilizing leader election on uniform rings. In *WDAG95 Distributed Algorithms 9th International Workshop Proceedings, Springer LNCS:972*, pages 288–302, 1995.
12. M. Gradinariu and C. Johnen. Self-stabilizing neighborhood unique naming under unfair scheduler. In *Euro-Par'01 Parallel Processing, Proceedings LNCS:2150*, pages 458–465, 2001.
13. W. Goddard, S. T. Hedetniemi, D. P. Jacobs, and Pradip K. Srimani. Anonymous daemon conversion in self-stabilizing algorithms by randomization in constant space. In *Proceedings of the 9th international conference on Distributed computing and networking, ICDCN'08*, pages 182–190, Berlin, Heidelberg, 2008. Springer-Verlag.
14. J. Beauquier, A.K. Datta, M. Gradinariu, and F. Magniette. Self-stabilizing local mutual exclusion and daemon refinement. In *DISC00 Distributed Computing 14th International Symposium, Springer LNCS:1914*, pages 223–237, 2000.
15. S.C. Hsu and S.T. Huang. A self-stabilizing algorithm for maximal matching. *Inform. Process. Lett.*, 43:77–81, 1992.
16. J. Beauquier, M. Gradinariu, and C. Johnen. Cross-over composition—enforcement of fairness under unfair adversary. In *WSS01 Proceedings of the Fifth International Workshop on Self-Stabilizing Systems, Springer LNCS:2194*, pages 19–34, 2001.
17. L. Lamport. A new solution of Dijkstra's concurrent programming problem. *Communications of the ACM*, 17:453–455, 1974.
18. B. Awerbuch, S. Kutten, Y. Mansour, B. Patt-Shamir, and G. Varghese. Time optimal self-stabilizing synchronization. In *STOC93 Proceedings of the 25th Annual ACM Symposium on Theory of Computing*, pages 652–661, 1993.
19. M.H. Karaata. Self-stabilizing strong fairness under weak fairness. *IEEE Trans. Parallel Distrib. Systems*, 12:337–345, 2001.
20. M.G. Gouda and F. Haddix. The alternator. In *Proceedings of the Fourth Workshop on Self-Stabilizing Systems (published in association with ICDCS99)*, pages 48–53. IEEE Computer Society, 1999.
21. M. Mizuno and H. Kakugawa. A timestamp based transformation of self-stabilizing programs for distributed computing environments. In *WDAG96 Distributed Algorithms 10th International Workshop Proceedings, Springer LNCS:1151*, pages 304–321, 1996.
22. H. Kakugawa, M. Mizuno, and M. Nesterenko. Development of self-stabilizing distributed algorithms using transformation: case studies. In *Proceedings of the Third Workshop on Self-Stabilizing Systems*, pages 16–30. Carleton University Press, 1997.
23. M. Nesterenko and A. Arora. Stabilization-preserving atomicity refinement. *J. Parallel Distrib. Comput.*, 62(5):766–791, 2002.
24. S. Shukla, D. Rosenkrantz, and S. Ravi. Developing self-stabilizing coloring algorithms via systematic randomization. In *Proceedings of the International Workshop on Parallel Processing*, pages 668–673, 1994.
25. J.H. Hoepman. Uniform deterministic self-stabilizing ring-orientation on odd-length rings. In *WDAG94 Distributed Algorithms 8th International Workshop Proceedings, Springer LNCS:857*, pages 265–279, 1994.
26. D. Angluin. Global and local properties in networks of processors. In *Proc. 12th Symposium on the Theory of Computing*, pages 82–93, 1980.